

Engineering Ingegneria Informatica

Enterprise Solution Center

ABAP Best Practice

Documento Tecnico

This document contains data of property of Engineering.IT and must be utilized exclusively from the addressee related to the beginning purpose of use. Every copy or spreading without Engineering.IT authorization is forbidden.

Written by:	Stefano Bertuzzi	3/08/2016
Verified by:	Insert Name	dd/mm/yyyy
Approved by:	Insert Name	dd/mm/yyyy

Summary

Questo documento illustra le best practice inerenti alla scrittura del codice ABAP adottate nella scrittura del codice custom da parte dell'Enterprise Solution Center di Engineering Ingegneria Informatica.

Distribution List

Nome	Ruolo
Stefano Bertuzzi	
...	

Document version

Tutte le modifiche sono riferite alle precedenti versioni

Version	Ref. prev. vers.
01 (03 agosto 2016)	

References

List of reference documents.

[1]

[2]

[3]

[4]

Indice

1.	Introduzione	4
2.	Indicazioni obbligatorie	5
2.1.	Critical statements	5
2.2.	Use of ADBC Interface	5
2.3.	DB Operation in Pool/Cluster Tables	5
2.4.	Problematic statements for results of SELECT/OPEN CURSOR without ORDER BY	5
2.5.	CALL FUNCTION 'DB_EXISTS_INDEX' * CALL FUNCTION 'DD_INDEX_NAME' *	5
3.	Indicazioni opzionali	6
3.1.	WHERE condition for SELECT, UPDATE and DELETE	6
3.2.	Table Attributes	6
3.3.	Unsecure use of FOR ALL ENTRIES	6
4.	Indicazioni inerenti il miglioramento delle performance	7
4.1.	SELECT statements that bypass the Table Buffer	7
4.2.	Problematic SELECT * statements	7
4.3.	SELECT ...FOR ALL ENTRIES clauses to be transformed	7
4.4.	EXIT or no statement in SELECT ...ENDSELECT loop	7
4.5.	SELECT statements with subsequent CHECKS	7

1. Introduzione

L'obiettivo di questo documento è quello di descrivere le principali linee guida da seguire nella scrittura del codice ABAP custom soprattutto in vista della migrazione alla piattaforma SAP HANA da parte di molti clienti del Gruppo Engineering.

Poiché durante una migrazione ad HANA la raccomandazione di SAP è quello di sottoporre a verifica il codice ABAP custom mediante il tool Code Inspector (transazione SCI) utilizzando tre varianti standard (FUNCTIONAL_DB, FUNCTIONAL_DB_ADDITION, PERFORMANCE_DB), le best practice descritte nei capitoli seguenti sono state derivate proprio da detti controlli.

La struttura di questo documento è divisa in 3 categorie:

- indicazioni obbligatorie (controllate dal Code Inspector mediante la variante FUNCTIONAL_DB)
- indicazioni opzionali ma fortemente consigliate (controllate dal Code Inspector mediante la variante FUNCTIONAL_DB_ADDITION)
- indicazioni inerenti il miglioramento delle performance (controllate dal Code Inspector mediante la variante PERFORMANCE_DB)

2. Indicazioni obbligatorie

2.1. Critical statements

Le operazioni sulle tabelle mediante SQL Nativo (EXEC ... ENDEXEC) oppure mediante database hint (%_HINTS) sono da evitare in quante sulla piattaforma HANA sono funzionalità deprecate oppure non più supportate.

2.2. Use of ADBC Interface

L'uso della classe CL_SDB_ADBC e simili è deprecato (ABAP Database Connectivity).

2.3. DB Operation in Pool/Cluster Tables

L'accesso diretto alle tabelle Pool o Cluster è deprecato (esempio: invece di leggere la tabella RFBLG si deve leggere la tabella BSEG).

2.4. Problematic statements for results of SELECT/OPEN CURSOR without ORDER BY

Le SELECT senza ORDER BY che caricano una tabella interna non sortata vanno evitate perché non c'è nessuna garanzia di ordinamento con HANA. Su database tradizione è preferibile effettuare il SORT della tabella interna. Su HANA è preferibile utilizzare la SELECT ... ORDER BY.

2.5. CALL FUNCTION 'DB_EXISTS_INDEX' * CALL FUNCTION 'DD_INDEX_NAME' *

Sono due Function Module non più presenti su HANA.

3. Indicazioni opzionali

3.1. WHERE condition for SELECT, UPDATE and DELETE

Evitare istruzioni SELECT/UPDATE/DELETE senza la condizione WHERE.

3.2. Table Attributes

Technical Settings

- Selezionare sempre la “Table Class” oppure la “Delivery Class”
- Selezionare sempre la “Data Class” oppure la “Size Category”
- Evitare di creare tabelle che abbiano più di 700 campi (in Account Payable più di 100 campi)
- Evitare di creare tabelle che contengano più di tre campi STRING oppure RAWSTRING

Table Indexes

- Evitare tabelle con indici secondari univoci
- Evitare di creare tabelle di Data Class “APPL0” con più di 4 indici secondari
- Evitare di creare tabelle di Data Class “APPL1” con più di 2 indici secondari
- Evitare di creare indici secondari con più di 4 campi
- Evitare di creare tabelle bufferizzate con indici secondari
- Evitare di creare tabelle con più di due campi uguali in due diversi indici
- Evitare di creare indici secondari con campi di tipo FLOAT
- Evitare di creare indici secondari su tabelle dipendenti dal mandante che non abbiano il mandante come primo campo

3.3. Unsecure use of FOR ALL ENTRIES

Controllare sempre la presenza di almeno un record nella tabella di check utilizzata dall’istruzione FOR ALL ENTRIES.

4. Indicazioni inerenti il miglioramento delle performance

4.1. SELECT statements that bypass the Table Buffer

Evitare di bypassare il buffer se una tabella è bufferizzata (istruzione SELECT ... BYPASSING BUFFER)

4.2. Problematic SELECT * statements

Evitare assolutamente di utilizzare SELECT * se poi si utilizzano un numero di campi inferiori al 20% del totale dei campi della tabella.

4.3. SELECT ...FOR ALL ENTRIES clauses to be transformed

Evitare di usare l'istruzione SELECT ... FOR ALL ENTRIES sostituendola con una SELECT ... join.

4.4. EXIT or no statement in SELECT ...ENDSELECT loop

Evitare assolutamente il loop SELECT ... ENDSELECT anche solo per verificare la presenza di un record nella tabella (in questo caso è preferibile usare la "UP TO 1 ROWS")

4.5. SELECT statements with subsequent CHECKS

Evitare di leggere dati per poi non utilizzarli.